Principle and Practice of Bidirectional Programming in BiGUL

Zhenjiang Hu, Hsiang-Shang Ko

National Institute of Informatics, Japan
{hu,hsiang-shang}@nii.ac.jp

Abstract. Putback-based bidirectional programming allows the programmer to write only one putback transformation, from which the unique corresponding forward transformation is derived for free. A key distinguishing feature of putback-based bidirectional programming is its full control over the bidirectional behavior, which is important to specify intended bidirectional transformations without any ambiguity.
In this tutorial, we will introduce BiGUL, a simple but powerful putback-based bidirectional programming language, explaining its principle, showing how to develop various of bidirectional transformations in BiGUL, demonstrating its application to adaptive software development, and highlighting important issues and challenges for future work.

1 Putback-based Bidirectional Programming

A *bidirectional transformation* basically consists of a pair of transformations: the *forward* transformation *get* s is used to produce a target view v from a source s, while the *putback* transformation *put* s v is used to reflect modifications on the view v to the source s. These two transformations should be *well-behaved* in the sense that they satisfy the following round-tripping laws.

$$put\ s\ (get\ s) = s \qquad \textsc{GetPut}$$
$$get\ (put\ s\ v) = v \qquad \textsc{PutGet}$$

The GetPut property requires that no changing on the view shall be reflected as no changing on the source, while the PutGet property requires all changes in the view to be completely reflected to the source so that the changed view can be computed again by applying the forward transformation to

Bidirectional programming is to develop well-behaved bidirectional transformations (BXs) to solve various synchronization problems. A straightforward approach to bidirectional programming is to write two unidirectional transformations. Although this ad-hoc solution provides full control over both get and putback transformations and can be realized using standard programming languages, the programmer needs to show that the two transformations satisfy the well-behavedness laws, and a modification to one of the transformations requires a redefinition of the other transformation as well as a new well-behavedness proof.

To ease and enable maintainable bidirectional programming, it is preferable to write just a single program that can denote both transformations.

Lots of work [1, 2, 6–8, 10, 13, 14] has been devoted to the *get-based* approach, allowing users to write the forward transformation *get* and deriving a suitable putback transformation. While the get-based approach is friendly, a *get* function may not be injective, so there may exist many possible *put* functions that can be combined with it to form a valid BX and there is no way to control the choice of *put* through the change of *get*. This ambiguity of put is what makes bidirectional programming challenging and unpredictable in practice.

The main topic of this tutorial is the *putback-based* approach to bidirectional programming. In contrast to the get-based approach, it allows users to write the backward transformation *put* and derives a suitable *get* that can be paired with *put* to form a bidirectional transformation if it exists. Interestingly, while *get* usually loses information when mapping from a source to a view, *put* must preserve information when putting back from the view to the source, according to the PutGet property.

Before explaining how to write *put*, let us briefly review the foundation from [3–5] showing that putback is the essence of bidirectional programming. We start by defining validity of *put* as follows.

Definition 1 (Validity of *put*). *We say that a put is* valid *if there exists a get such that both* GetPut *and* PutGet *are satisfied.*

The first interesting fact is that, for a valid *put*, there exists at most one *get* that can form a BX with it. This is in sharp contrast to get-based bidirectional programming, where many *put*s can be paired with a *get* to form a BX.

Lemma 1 (Uniqueness of *get*). *Given a put function, there exists at most one get function that forms a well-behaved BX.*

The second interesting fact is that it is possible to check validity *put* without mentioning *get*. The following two properties on *put* whose combination is equivalent to GetPut and PutGet.

- The first, that we call *view determination*, says that equivalence of updated sources produced by a *put* implies equivalence of views that are put back.

$$\forall\ s, s', v, v'.\ put\ s\ v\ =\ put\ s'\ v'\ \Rightarrow\ v\ =\ v' \qquad \textsc{ViewDetermination}$$

  Note that the view determination implies that $put\ s$ is injective (with $s = s'$).
- The second, that we call *source stability*, denotes a slightly stronger notion of surjectivity for every source:

$$\forall\ s.\ \exists\ v.\ put\ s\ v\ =\ s \qquad \textsc{SourceStability}$$

Actually, these two laws together provide an equivalent characterization of the validity of *put*.

Theorem 1. *A put function is valid if and only if it satisfies the* ViewDetermination *and* SourceStability *properties.*

Practically, however, there have not been many researches on put-based bidirectional programming. This is not without reason: as argued in [5], it is far from being straightforward to construct a framework that can directly support putback-based bidirectional programming.

This tutorial introduces BiGUL [9], a simple but powerful putback-based bidirectional language, which which grew out of the work [11,12]. We will demonstrate how to develop bidirectional programs in BiGUL, explains the principle behind BiGUL, and show its applications in developing various of bidirectional transformations.

2 Preparation: Installing BiGUL

The latest version of BiGUL has been released to Hackage. To install the package, just run the following two commands.

```
> cabal update
> cabal install bigul
```

Now you can easily check whether BiGUL is correctly installed. First, create a simple file called "test.hs" with the following content for importing BiGUL modules.

```
import Generics.BiGUL
import Generics.BiGUL.TH
import Generics.BiGUL.Lib
```

Then load it using the GHC interactive interpreter.

```
> ghci test.hs
GHCi, version 7.10.3: http://www.haskell.org/ghc/  :? for help
Prelude>~/Repositories/prl_tokyo/bigul/SummerSchool16> ghci test.hs
GHCi, version 7.10.3: http://www.haskell.org/ghc/  :? for help
[1 of 1] Compiling Main             ( test.hs, interpreted )
Ok, modules loaded: Main.
```

If you see the above message, congratulations on your successful installation.

3 A Quick Tour of BiGUL

Intuitively, we can think of a bidirectional BiGUL program:

$$bx :: BiGUL\ s\ v$$

as describing how to manipulate a state consisting of a source component of type s and a view component of type v; the goal is to embed all information in

the view to proper places in the source. For each bx, we can run it forwardly by calling *get* and backwardly by calling *put*.

$$get\ bx :: s \to v$$
$$put\ bx :: s \to v \to s$$

where *get bx* is a function mapping a source to a view, while *put bx* accepts an original source and an update view and returns an updated source.

The core of BiGUL consists of a small number of combinators for constructing bidirectional transformation through development of well-behaved putback transformations.

3.1 Skip

The first one is the simple primitive

$$Skip :: BiGUL\ s\ ()$$

Its putback semantics is to skip any change on the source for the unit view. One can test this using ghci after loading the test.hs by

```
*Main> put Skip 5 ()
Right 5
```

which means that *Skip* putbacks the unit view () on the source 5 and gets the same source 5. In fact, each well-behaved putback transformation is equipped with a unique *get* for doing forward transformation.

```
*Main> get Skip 5
Right ()
```

It reads that doing forward transformation for *Skip* on the source 5 gives the unit view ().

3.2 Replace

The second one is

$$Replace :: BiGUL\ s\ s$$

which is to use the view to replace the source:

```
*Main> put Replace 1 100
Right 100
```

which uses the view 100 to replace the source 1 and gets a new source 100.

3.3 Product: Prod

If we want to putback a view pair (v_1, v_2) to a source pair (s_1, s_2), we may write *bx1 Prod bx2*, a product of two putback transformation, to putback v_1 to s_1 with *bx1* and v_2 to s_2 with *bx2*.

$$Prod :: BiGUL\ s_1\ v_1 \rightarrow BiGUL\ s_2\ v_2 \rightarrow BiGUL\ (s_1, s_2)\ (v_1, v_2)$$

For instance, we can combine *Skip* and *Replace* to putback a view pair to a source pair.

```
*Main> put (Skip `Prod` Replace) (5,1) ((),100)
Right (5,100)
```

We may deal with more complicated structure using nested *Prod*:

```
*Main> put ((Skip `Prod` Replace) `Prod` Replace) ((5,1),2) (((),100),200)
Right ((5,100),200)
```

3.4 Source/View Rearrangement

So far, the source and the view must be of the same structure. What if we wish to putback a pair view (v_1, v_2) to a complicated source, say $((s0, s_1), s_2))$, where we hope to replace s_1 by v_1 and s_2 by v_2? BiGUL provide a way of prepossessing of rearranging either the source or the view through a natural transformation *tau*.

$$\$\,(rearrS\ [|\ tau :: s_1 \rightarrow s_2\ |]) :: BiGUL\ s_2\ v \rightarrow BiGUL\ s_1\ v$$
$$\$\,(rearrV\ [|\ tau :: v_1 \rightarrow v_2\ |]) :: BiGUL\ s\ v_2 \rightarrow BiGUL\ s\ v_1$$

Returning to the above problem, we can solve it by defining the following putback transformation

$$putPairOverNPair :: BiGUL\ ((s0, s_1), s_2)\ (s_1, s_2)$$
$$putPairOverNPair = \$(rearrS\ [|\ \lambda((s0, s_1), s_2) \rightarrow (s_1, s_2)\ |])\ Replace$$

and have

```
*PBasic> put putPairOverNPair ((5,1),2) (100,200)
Right ((5,100),200)
```

In fact, it is possible to define *putPairOverNPair* by rearranging the view too.

$$putPairOverNPair' :: BiGUL\ ((s0, s_1), s_2)\ (s_1, s_2)$$
$$putPairOverNPair' = \$\,(rearrV\ [|\ \lambda(v_1, v_2) \rightarrow (((), v_1), v_2)\ |])\ \$$$
$$(Skip\ `Prod`\ Replace)\ `Prod`\ Replace$$

The mechanism of source/view rearrangement makes it possible to process algebraic data structures such as lists and trees. The following example describes using view v to replace the first element of a nonempty source.

$$pHead :: BiGUL\ [s]\ s$$
$$pHead = \$(rearrS\ [|\ \lambda(s{:}\ \_) \rightarrow s\ |])\ Replace$$

```
*PBasic> put pHead [1,2,3,4] 100
Right [100,2,3,4]
```

We may go further to define a general putback transformation to replace the ith element of a source list with a view v.

$$
\begin{aligned}
&pNth :: \mathbb{Z} \to BiGUL\ [s]\ s \\
&pNth\ i = \textbf{if}\ i \equiv 0\ \textbf{then}\ pHead \\
&\qquad \textbf{else}\ \$\ (rearrS\ [|\ \lambda(x : xs) \to (x, xs)\ |])\ \$ \\
&\qquad\qquad \$\ (rearrV\ [|\ \lambda v \to ((), v)\ |])\ \$ \\
&\qquad\qquad\quad Skip\ `Prod`\ pNth\ (i - 1)
\end{aligned}
$$

```
*PBasic> put (pNth 3) [1..10] 100
Right [1,2,3,100,5,6,7,8,9,10]
```

Note that any putback function is equipped with a *get* function (which may be partial). So for *pNth*, the corresponding *get* function is actually the familiar *take* function.

```
*PBasic> get (pNth 3) [1..10]
Right 4
```

3.5 Case

The *Case* combinator is for case analysis. The basic structure is as follows.

$$
\begin{aligned}
&Case\ [\ \$\ (normal\ [|\ cond1 :: s \to v \to Bool\ |])\ (bx1 :: BiGUL\ s\ v), \\
&\quad\ \$\ (normal\ [|\ cond2 :: s \to v \to Bool\ |])\ (bx2 :: BiGUL\ s\ v), \\
&\quad\ ..., \\
&\quad\ \$\ (normal\ [|\ condn :: s \to v \to Bool\ |])\ (bxn :: BiGUL\ s\ v), \\
&\quad\ \$\ (adaptive\ [|\ cond1' :: s \to v \to Bool\ |])\ \$\ (f1 :: s \to v \to s), \\
&\quad\ \$\ (adaptive\ [|\ cond2' :: s \to v \to Bool\ |])\ \$\ (f2 :: s \to v \to s), \\
&\quad\ ..., \\
&\quad\ \$\ (adaptive\ [|\ condm' :: s \to v \to Bool\ |])\ \$\ (fm :: s \to v \to s) \\
&\quad\ ] \\
&\ :: BiGUL\ s\ v
\end{aligned}
$$

It contains a sequence of cases. For each case, it is either *normal* or *adaptive*. For the normal case, if the condition is satisfied, a corresponding putback transformation is applied. For the adaptive case, if the condition is satisfied, a function is used to update the source with the view so that for the next step one of the normal cases can be applied. Note that if adaptation does not lead the source and the view to a normal case, an error will be reported at runtime.

As a simple example, consider using the view to update all the elements in the source list. To do so, we can use *Case*.

$$
\begin{aligned}
&replaceAll :: Eq\ s \Rightarrow BiGUL\ [s]\ s \\
&replaceAll = Case\ [
\end{aligned}
$$

```
    $ (normal [| λs v → length s ≡ 1 |]) $
      $ (rearrS [| λ[x] → x |]) Replace,
    $ (normal [| λs v → length s > 1 |]) $
      $ (rearrS [| λ(x : xs) → (x, xs) |]) $
        $ (rearrV [| λv → (v, v) |]) $
          Replace `Prod` replaceAll,
    $ (adaptive [| λs v → length s ≡ 0 |]) $
      λs v → [⊥]
]
```

```
*PBasic> put replaceAll [] 100
Right [100]
*PBasic> put replaceAll [1..10] 100
Right [100,100,100,100,100,100,100,100,100,100]
```

3.6 Dependency: Dep

Sometimes, a view may contain derived value that is computed from other part of the view and not allowed to be changed. For instance, for the view $(x, x+1)$, the second component can be computed from the first by increasing it by 1. To capture this, BiGUL provides

$$Dep :: Eq\ v' \Rightarrow BiGUL\ a\ v \to (a \to v \to v') \to BiGUL\ a\ (v, v')$$

to describe this intention. We may, for example, define

```
replaceAll2 :: BiGUL [ℤ] (ℤ, ℤ)
replaceAll2 = Dep replaceAll (λs v → v + 1)
```

to replace all elements of the source by the first component of the view, while the second component of the view is a derived one that should be one bigger than the first one.

```
*PBasic> put replaceAll2 [1..10] (100,101)
Right [100,100,100,100,100,100,100,100,100,100]
*PBasic> put replaceAll2 [1..10] (100,200)
Left dependency mismatch
```

As seen in the last running of *put*, it reports an error because in the view $(100, 200)$, 200 is not one bigger than 100.

3.7 Composition

Composition of two putback transformations follows that of two bidirectional transformations This is because each putback transformation is essentially a bidirectional transformation.

$$Compose :: BiGUL\ a\ u \to BiGUL\ u\ b \to BiGUL\ a\ b$$

As an example, consider

$$pHead2 :: BiGUL\ [[a]]\ a$$
$$pHead2 = pHead\ `Compose`\ pHead$$

```
*PBasic> put pHead2 [[1,2],[3,4,5],[]] 100
Right [[100,2],[3,4,5],[]]
```

3.8 Utilities

BiGUL provides some useful functions for building putback transformations. One interesting one is *emb* that can safely embed a pair of well-behaved *get* and *put* a putback transformation into our context. *emb* itself is defined as follows.

$$\begin{aligned}
&emb :: Eq\ v \Rightarrow (s \to v) \to (s \to v \to s) \to BiGUL\ s\ v\\
&emb\ g\ p = Case\ [\ \$\ (normal\ [|\ \lambda x\ y \to g\ x \equiv y\ |])\ \$\\
&\qquad\qquad \$\ (rearrV\ [|\ \lambda x \to ((), x)\ |])\ \$\\
&\qquad\qquad\quad Dep\ Skip\ (\lambda x\ () \to g\ x)\\
&\qquad\quad ,\ \$\ (adaptive\ [|\ \backslash__ \to True\ |])\\
&\qquad\qquad p\\
&\qquad\quad]
\end{aligned}$$

As an application of *emb*, we may define a useful putback functions for reflecting the sum to a pair.

$$\begin{aligned}
&distSum :: BiGUL\ (\mathbb{Z}, \mathbb{Z})\ \mathbb{Z}\\
&distSum = emb\ g\ p\\
&\quad \textbf{where}\ g\ (x, y) = x + y\\
&\qquad\quad p\ (x, y)\ v = (v - y, y)
\end{aligned}$$

```
*PBasic> put distSum (1,2) 100
Right (98,2)
*PBasic> get distSum (1,2)
Right 3
```

4 Bidirectional Programming on Lists

In this section, we show many list functions can be bidirectionalized using BiGUL. To show the correspondence with the original we prefix the original forward function names with "lens". Note that the forward function can be automatically derived from the new putback transformation by calling *get*.

4.1 lensFoldr

foldr is a simple but useful higher order function on lists.

$$
\begin{aligned}
&\mathit{foldr}\ f\ e\ [\,] &&= e\\
&\mathit{foldr}\ f\ e\ (x : xs) &&= f\ x\ (\mathit{foldr}\ f\ e\ xs)
\end{aligned}
$$

Many intereting functions can be defined in *foldr*

$$
\begin{aligned}
&\mathit{sum} = \mathit{foldr}\ (+)\ 0\\
&\mathit{map}\ f = \mathit{foldr}\ (\lambda a\ r \to f\ a : r)\ [\,]\\
&\mathit{filter}\ p = \mathit{foldr}\ (\lambda a\ r \to \mathbf{if}\ p\ a\ \mathbf{then}\ a : r\ \mathbf{else}\ r)\ [\,]\\
&\mathit{reverse}\ p = \mathit{foldr}\ (\lambda a\ r \to r +\!\!+ [a])\ [\,]\\
&\mathit{sort} = \mathit{foldr}\ \mathit{insert}\ [\,]
\end{aligned}
$$

First, we develop a putback transformation for *foldr*.

$$
\begin{aligned}
&\mathit{lensFoldr} :: \mathit{BiGUL}\ (a, b)\ b \to (b \to \mathit{Bool}) \to \mathit{BiGUL}\ ([a], b)\ b\\
&\mathit{lensFoldr}\ \mathit{bx}\ \mathit{pv} =\\
&\quad \mathit{Case}\ [\ \$\ (\mathit{adaptive}\ [|\ \lambda(x, y)\ v \to \mathit{pv}\ v \wedge \mathit{length}\ x \not\equiv 0\ |])\ \$\\
&\qquad\qquad \lambda(x, y)\ v \to ([\,], y)\\
&\qquad , \$\ (\mathit{normalS}\ [|\ \lambda(s, e) \to \mathit{length}\ s \equiv 0\ |])\ \$\\
&\qquad\qquad \$\ (\mathit{rearrV}\ [|\ \lambda v \to ((), v)\ |])\ \$\\
&\qquad\qquad\quad \$\ (\mathit{update}\ [p \mid ((), v)\ |]\ [p \mid (\_, v)\ |]\ [d \mid v = \mathit{Replace}\ |])\\
&\qquad , \$\ (\mathit{normalSV}\ [p \mid \_\ |]\ [p \mid \_\ |])\ \$\\
&\qquad\qquad \$\ (\mathit{rearrS}\ [|\ \lambda((x : xs), e) \to (x, (xs, e))\ |])\\
&\qquad\qquad\quad (\mathit{Replace}\ `\mathit{Prod}`\ \mathit{lensFoldr}\ \mathit{bx}\ \mathit{pv})\ `\mathit{Compose}`\ \mathit{bx}\\
&\qquad]
\end{aligned}
$$

lengFoldr bx pv is to synchronize source (xs, v) with view v'. If v' satisfies *pv*, it will stop. Otherwise, it will recursively apply *bx* rightwards over the elements of *xs*.

$$
\begin{aligned}
&\mathit{lensReverse} :: \mathit{BiGUL}\ [a]\ [a]\\
&\mathit{lensReverse} = \ \$\ (\mathit{rearrS}\ [|\ \lambda s \to (s, [\,])\ |])\ \$\\
&\qquad\qquad\qquad \mathit{lensFoldr}\ \mathit{lensSnoc}\ \mathit{null}\\
&\mathit{lensSnoc} :: \mathit{BiGUL}\ (a, [a])\ [a]\\
&\mathit{lensSnoc} = \mathit{Case}\ [\ \$\ (\mathit{normal}\ [|\ \lambda s\ v \to \mathit{length}\ v \equiv 1\ |])\ \$\\
&\qquad\qquad\qquad\qquad \$\ (\mathit{rearrV}\ [|\ \lambda[v] \to (v, [\,])\ |])\ \mathit{Replace},\\
&\qquad\qquad\qquad \$\ (\mathit{normal}\ [|\ \lambda(\_, s)\ v \to \mathit{length}\ s > 0\ |])\ \$\\
&\qquad\qquad\qquad\quad \$\ (\mathit{rearrS}\ [|\ \lambda(x, y : ys) \to (y, (x, ys))\ |])\ \$\\
&\qquad\qquad\qquad\qquad \$\ (\mathit{rearrV}\ [|\ \lambda(v : vs) \to (v, vs)\ |])\ \$\\
&\qquad\qquad\qquad\qquad\quad \mathit{Replace}\ `\mathit{Prod}`\ \mathit{lensSnoc},\\
&\qquad\qquad\qquad \$\ (\mathit{adaptive}\ [|\ \lambda(\_, s)\ v \to \mathit{null}\ s\ |])\ \$\\
&\qquad\qquad\qquad\quad \lambda(x, s)\ \_ \to (x, [\bot])\\
&\qquad\qquad\qquad]
\end{aligned}
$$

```
*PList> put lensSnoc (1,[2,3,4]) [10,11,12,13]
Right (13,[10,11,12])
*PList> put lensSnoc (1,[2,3,4]) [10,11,12,13,14]
Right (14,[10,11,12,13])
*PList> put lensSnoc (1,[2,3,4]) [10,11]
Right (11,[10])
*PList> get lensSnoc (100,[1..10])
Right [1,2,3,4,5,6,7,8,9,10,100]

*PList> put lensReverse [1..10] [100..105]
Right [105,104,103,102,101,100]
*PList> put lensReverse [1..10] [100..115]
Left either value mismatch
*PList> get lensReserve [1..10]
*PList> get lensReverse [1..10]
Right [10,9,8,7,6,5,4,3,2,1]
```

4.2 Efficiency Issue: *lensFoldr*

A close look at the definition of *lensFoldr* reveals that it contains many redundent computations because of the use of *Compose* that calls *get* as many times as the number of calls of *Compose*. Put it more concretely,

$$put\ (lensFoldr\ bx\ (const\ True))\ ([x1, x2, ..., xn], e)\ v$$

would call

$$get\ (lensFoldr\ bx\ (const\ True))\ ([x2, ..., xn], e),$$
$$...,$$
$$get\ (lensFoldr\ bx\ (const\ True))\ ([\,], e)\ \circ$$

4.3 LensScanr

4.4 LensMap

$$lensMap :: BiGUL\ a\ b \to BiGUL\ ([a], [b])\ [b]$$
$$lensMap\ bx = lensFoldr\ bx'\ (\lambda v \to length\ v \equiv 0)$$
$$\textbf{where}\ bx' = \$\ (rearrV\ [|\ \lambda(v : vs) \to (v, vs)\ |])\ \$$$
$$bx\ `Prod`\ Replace$$

For testing, we define

$$dec1 :: BiGUL\ \mathbb{Z}\ \mathbb{Z}$$
$$dec1 = emb\ g\ p$$
$$\textbf{where}\ g\ s = s + 1$$
$$p\ s\ v = v - 1$$

```
*PList> put (lensMap dec1) ([0..10],[]) [100..110]
Right ([99,100,101,102,103,104,105,106,107,108,109],[])
*PList> get (lensMap dec1) ([1..10],[])
Right [2,3,4,5,6,7,8,9,10,11]
*PList> put (lensMap dec1) ([0..10],[]) [100]
Right ([99],[])
*PList> put (lensMap dec1) ([0..10],[]) [100..111]
Right ([99,100,101,102,103,104,105,106,107,108,109],[111])
```

5 Into BiGUL's Bidirectionality

We have been writing *put* programs, usually having a corresponding *get* in mind but not explicitly describing it, and yet BiGUL is capable of finding the right *get* behaviour as if reading our mind. How? We will see that, when writing a BiGUL program, we are always simultaneously describing both a *put* function and a *get* function, which are guaranteed to be a well-behaved pair. And the "mind-reading" ability is far from magic: Well-behavedness directly implies that *get* is uniquely determined by *put*, which is the main motivation for designing a putback-based language. We will look at several constructs of BiGUL in detail to get a taste of such design.

5.1 Lenses, well-behavedness, and the fundamental theorem

Formally, we call a well-behaved pair of *put* and *get* a *lens*:

Definition 2 (lens). *A* lens *between a source type s and a view type v consists of two functions:*

$$\begin{aligned} &put :: s \to v \to Maybe\ s \\ &get :: s \quad\ \to Maybe\ v \end{aligned}$$

satisfying two well-behavedness laws:

$$put\ s\ v = Just\ s' \quad \Rightarrow \quad get\ s' = Just\ v \qquad \text{(PutGet)}$$

$$get\ s = Just\ v \quad \Rightarrow \quad put\ s\ v = Just\ s \qquad \text{(GetPut)}$$

In the original formulation [6], a lens refers to just a pair of functions having the right types, and one needs to explicitly say "well-behaved lens" to mean a well-behaved pair; we will talk about well-behaved lenses only, though, so we build well-behavedness into our definition of lenses. Also note that this definition models partial transformations explicitly as *Maybe*-valued functions: *put* and *get* are *total* functions that can nevertheless produce *Nothing* to indicate failure.

From this definition of well-behavedness, we can immediately prove what might be called the "fundamental theorem" of putback-based bidirectional programming:

Theorem 2 (uniqueness of *get*). *Given two lenses whose put components are equal, their get components are also equal.*

Proof. Let l and r be two lenses; denote their *put*/*get* components as $put\ l/get\ l$ and $put\ r/get\ r$ respectively, and assume that $put\ l = put\ r$. Then for any s and v,

$$\begin{aligned} & get\ l\ s = Just\ v \\ \Leftrightarrow\quad & \{\,\text{well-behavedness of } l\,\} \\ & put\ l\ s\ v = Just\ s \\ \Leftrightarrow\quad & \{\,put\ l = put\ r\,\} \\ & put\ r\ s\ v = Just\ s \\ \Leftrightarrow\quad & \{\,\text{well-behavedness of } r\,\} \\ & get\ r\ s = Just\ v \end{aligned}$$

(This also entails that $get\ l\ s = Nothing$ if and only if $get\ r\ s = Nothing$.) □

The theorem guarantees that the BiGUL programmer is in full control of the bidirectional behavior — programming the *put* behavior is sufficient to determine the *get* behavior. Also, to the language designer, the theorem gives a kind of reassurance that, once the *put* behavior of a construct is determined, there is no need to worry about which *get* behavior should be adopted — there is at most one possibility. This is in contrast to *get*-based design, in which there are usually more than one viable *put* semantics that can be assigned to a *get*-based construct, and the designer needs to justify the choice or provide several versions.

5.2 Designing putback-based language components

Each BiGUL construct is conceived, at the design stage, as a lens (like *Skip* and *Replace*) or a lens *combinator* (like *Case*), which constructs a more complex lens from simpler ones. The *put* and *get* components of these lenses usually have to be developed together, but for each lens we will employ a more "*put*-oriented" design process: We start from an intended *put* behavior, and then add restrictions so that we can find a corresponding *get*. This does not guarantee that the lenses we arrive at will have a "strong *put* flavor" — that is, some of the lenses will be as suitable for *get*-based programming as for putback-based programming (or even more suitable). But we will also see that some other lenses are more naturally understood in terms of their *put* behavior.

Replace. The simplest lens is probably *Replace*, which replaces the entire source with the view:

$$put\ Replace\ s\ v = Just\ v$$

Is there a *get* semantics that can be paired with this *put*? Yes, quite obviously — in fact, PutGet directly gives us the definition of *get Replace*:

$$get\ Replace\ v = Just\ v$$

We still need to verify GetPut, which can be easily checked to be true.

Skip. Coming up next is *Skip*, whose natural behavior is

$$put\ Skip\ s\ v = Just\ s$$

Considering PutGet, though, we immediately see that this behavior is too liberal: If the view is simply thrown away, how can *get Skip* possibly recover it? One way out is to require that the view is trivial enough such that it can be thrown away and still be recovered, by setting the view type of *Skip* to the unit type (). Then it is easy for *get Skip* to recover the view, for which there is only one choice:

$$get\ Skip\ s = Just\ ()$$

This is the approach adopted prior to BiGUL 1.0.

More generally, we can establish well-behavedness as long as *get Skip* has only one view choice for each source, regardless of what the view type is. The existence of this "unique choice" is witnessed by a function $f :: s \to v$, which we add as an additional argument to *Skip*. The *get* direction is then

$$get\ (Skip\ f)\ s = Just\ (f\ s)$$

From the *put* direction, we may think of this function f as specifying a consistency relation, saying that the view information is completely included in the source (since you can compute the view from the source) and can be safely discarded. *Skip f* can be used if and only if the source and view are consistent in that sense. We thus arrive at:

$$put\ (Skip\ f)\ s\ v = \textbf{if}\ v \equiv f\ s\ \textbf{then}\ return\ s\ \textbf{else}\ Nothing$$

This pair of *put* and *get* can be verified to be well-behaved. *Skip f*, which features in BiGUL 1.0, is one lens which turns out to be more easily understood from the *get* direction — it bidirectionalizes any function whose codomain has decidable equality, albeit trivially. We get the first version of *Skip* as a special case by setting f to *const* ().

Prod. For a simplest example of a lens combinator, we look at *Prod*. Both the source and view types should be pairs; *Prod* accepts two lenses, say l and r, and applies them respectively to the left and right components:

$$\begin{aligned} put\ (l\ `Prod`\ r)\ (sl, sr)\ (vl, vr) = \textbf{do}\ & sl' \leftarrow put\ l\ sl\ vl \\ & sr' \leftarrow put\ r\ sr\ vr \\ & return\ (sl', sr') \end{aligned}$$

The *get* direction is unsurprising:

$$\begin{aligned} get\ (l\ `Prod`\ r)\ (sl, sr) = \mathbf{do}\ & vl \leftarrow get\ l\ \ sl \\ & vr \leftarrow get\ r\ sr \\ & return\ (vl, vr) \end{aligned}$$

Having constructed *put* and *get* from l and r, we also expect that their well-behavedness is a consequence of the well-behavedness of l and r. Intuitively, this pair of *put* and *get* does look well-behaved if l and r are. But how do we establish that formally? To prove PutGet, for example, we should prove from the assumption

$$put\ (l\ `Prod`\ r)\ (sl, sr)\ (vl, vr) = Just\ (sl', sr') \tag{1}$$

the conclusion

$$get\ (l\ `Prod`\ r)\ (sl', sr') = Just\ (vl, vr) \tag{2}$$

Both equations say that a somewhat complicated monadic *Maybe*-program computes successfully to some value. It may seem that we need some messy case analysis, but what we know about *Maybe*-programs tells us that such a program computes successfully if and only if every step of the program does, and this helps us to break both (1) and (2) into simpler equations. Formally, we have this lemma:

Lemma 2. *Let $mx :: Maybe\ a$ and $f :: a \to Maybe\ b$. Then, for all $y :: b$,*

$$mx \gg\!= f = Just\ y$$

if and only if

$$mx = Just\ x \quad and \quad f\ x = Just\ y \qquad for\ some\ x :: a$$

Proof. Case analysis on mx. □

This lemma can be nicely applied to *Maybe*-programs written in the **do**-notation, transforming such programs into *predicates* saying that a program computes to some given value. To do it more formally: Define a translation $\mathcal{S}$ from **do**-blocks of type $Maybe\ a$ to predicates on a by

$$\begin{aligned} \mathcal{S}\ (\mathbf{do}\ \{x \leftarrow mx; B\})\ y &= \exists x.\ mx = Just\ x \wedge \mathcal{S}\ (\mathbf{do}\ B)\ y \\ \mathcal{S}\ (\mathbf{do}\ \{my\})\ y &= my = Just\ y \end{aligned}$$

Then we can extend Lemma 2 to the following:

Lemma 3. *The proposition*

$$\mathcal{S}\ (\mathbf{do}\ B)\ y$$

is true if and only if

$$\mathbf{do}\ B = Just\ y$$

Proof. By induction on the list structure of B, using Lemma 2 repeatedly. □

For example, applying $\mathcal{S}$ to $put\ (l\ `Prod`\ r)\ (sl, sr)\ (vl, vr)$ yields

$$\begin{aligned}\lambda(sl'', sr''). \exists sl'.\ & put\ l\ sl\ vl = Just\ sl' \wedge \\ \exists sr'.\ & put\ r\ sr\ vr = Just\ sr' \wedge \\ & return\ (sl', sr') = Just\ (sl'', sr'')\end{aligned}$$

where the last equation is equivalent to $sl' = sl'' \wedge sr' = sr''$ (since $return = Just$ for the *Maybe* monad). Applying Lemma 3 and doing some simplification, (1) is equivalent to

$$put\ l\ sl\ vl = Just\ sl' \quad \wedge \quad put\ r\ sr\ vr = Just\ sr'$$

Similarly, (2) can be shown to be equivalent to

$$get\ l\ sl' = Just\ vl \quad \wedge \quad get\ r\ sr' = Just\ vr$$

The entailment is then just PutGet for l and r.

Case. This is a representative combinator in BiGUL, and arguably the most complex one. For simplicity, let us consider a two-branch variant of *Case* first. A branch is a condition and a body; since in *put* we manipulate both a source and a view, the conditions in general can be binary predicates on both the source and view. We thus define the type of branches as:

$$\textbf{type}\ CaseBranch\ s\ v = (s \to v \to Bool, BiGUL\ s\ v)$$

and consider the following variant of *Case*:

$$Case :: CaseBranch\ s\ v \to CaseBranch\ s\ v \to BiGUL\ s\ v$$

The straightforward behavior is

$$\begin{aligned}put\ (Case\ (pl, l)\ (pr, r))\ s\ v = \ & \textbf{if}\ \ pl\ \ s\ v\ \textbf{then}\ put\ l\ \ s\ v \\ & \textbf{else if}\ pr\ s\ v\ \textbf{then}\ put\ r\ s\ v \\ & \textbf{else}\ Nothing\end{aligned}$$

That is, depending on which condition is satisfied (with pl having higher priority), we execute either $put\ \ l$ or $put\ \ r$, or fail the computation if neither of the conditions is satisfied. Now, again, we ask the question: Can we find a *get* behavior to pair with this *put*?

Ruling out branch switching for PutGet. An important working assumption here is that we want lens combinators to be *compositional*: When we looked at *Prod*, for example, we defined its *put* and *get* in terms of those of the smaller lenses, and derived the overall well-behavedness from that of the smaller lenses. For *Case*, this implies that, when establishing well-behavedness, we want a *get* following a *put* (or a *put* following a *get*) to use the same branch taken by the

put (or the *get*), so we can invoke PutGet (or GetPut) of the branch. The current *put* behavior of *Case* does not leave any clue in the updated source about which branch is used to produce it, though, so it is impossible for *get* to choose the right branch.

One solution, which does not require changing the syntax of *Case*, is to check that the ranges of the branches are *disjoint*. In general, for a lens, the range of a *put* can be shown to coincide with the domain of the corresponding *get*. So the *get* behavior of *Case* can simply try to execute both branches on the input source, and there will be at most one branch that computes successfully. We can put (expensive) disjointness checks into *put* such that if *put* succeeds, the subsequent *get* will have at most one branch to choose:

$$
\begin{array}{l}
put\ (Case\ (pl, l)\ (pr, r))\ s\ v = \\
\quad \textbf{if} \quad\ pl\ s\ v\ \textbf{then do}\ s' \leftarrow put\ l\ \ s\ v \\
\qquad\qquad\qquad\qquad\quad maybe\ (return\ s')\ (const\ Nothing)\ (get\ r\ s') \\
\quad \textbf{else if}\ pr\ s\ v\ \textbf{then do}\ s' \leftarrow put\ r\ s\ v \\
\qquad\qquad\qquad\qquad\quad maybe\ (return\ s')\ (const\ Nothing)\ (get\ l\ \ s') \\
\quad \textbf{else}\ Nothing
\end{array}
$$

If *get* favors the first branch, meaning that it declares success as soon as the first branch succeeds (without requiring that the second branch fails),

$$get\ (Case\ (pl, l)\ (pr, r))\ s = maybe\ (get\ r\ s)\ return\ (get\ l\ s)$$

then we can also omit the check in *put*'s first branch:

$$
\begin{array}{l}
put\ (Case\ (pl, l)\ (pr, r))\ s\ v = \\
\quad \textbf{if} \quad\ pl\ s\ v\ \textbf{then}\ put\ l\ s\ v \\
\quad \textbf{else if}\ pr\ s\ v\ \textbf{then do}\ s' \leftarrow put\ r\ s\ v \\
\qquad\qquad\qquad\qquad\quad maybe\ (return\ s')\ (const\ Nothing)\ (get\ l\ s') \\
\quad \textbf{else}\ Nothing
\end{array}
$$

Ruling out branch switching for GetPut. The GetPut direction, on the other hand, still does not avoid branch switching — the outcome of *get* does not say anything about which of *pl* and *pr* will be satisfied in the subsequent *put*. So we add some checks to *get* such that *get*'s success will tell us which branch will be chosen by *put*:

$$
\begin{array}{l}
get\ (Case\ (pl, l)\ (pr, r))\ s = maybe\ (\textbf{do}\ v \leftarrow getBranch\ (pr, r)\ s \\
\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\quad \textbf{if}\ pl\ s\ v\ \textbf{then}\ Nothing \\
\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\quad \textbf{else}\ \ return\ v) \\
\qquad\qquad\qquad\qquad\qquad\quad return \\
\qquad\qquad\qquad\qquad\qquad\quad (getBranch\ (pl, l)\ s) \\
getBranch\ (pl, l)\ s = \textbf{do}\ v \leftarrow get\ l\ s \\
\qquad\qquad\qquad\qquad\quad \textbf{if}\ pl\ s\ v\ \textbf{then}\ \ return\ v \\
\qquad\qquad\qquad\qquad\qquad\quad \textbf{else}\ \ \ Nothing
\end{array}
$$

The definition of *put* should also be revised to use *getBranch* for disjointness check. But this breaks PUTGET! Since *put* does not guarantee that the updated source and the view satisfy the condition of the branch executed, even though *get* will be able to choose the right branch, the subsequent, newly added check is not guaranteed to succeed. We thus also need to add similar checks to *put*:

```
put (Case (pl, l) (pr, r)) s v =
  if      pl s v then do s' ← put l s v
                         if pl s' v then return s'
                                    else Nothing
  else if pr s v then do s' ← put r s v
                         if pr s' v then maybe (return s')
                                               (const Nothing)
                                               (getBranch (pl, l) s')
                                    else Nothing
  else Nothing
```

Now this pair of *put* and *get* can be verified to be well-behaved.

Improving the efficiency of get. The efficiency of the current *get* does not seem very good, especially when, in general, any number of branches is allowed, and *get* has to try to execute each branch, possibly with a high cost, until it reaches a successful one; also, inefficient *get* affects the efficiency of *put*, which calls *get* to check range disjointness. An idea is to ask the programmer to make a rough "prediction" of the range of each branch: We enrich *CaseBranch* with a third component, which is a source predicate:

```
type CaseBranch s v = (s → v → Bool, BiGUL s v, s → Bool)
```

This new predicate is supposed to be satisfied by the updated source; we again add checks to *put* to ensure this:

```
put (Case (pl, l, ql) (pr, r, qr)) s v =
  if      pl s v then do s' ← put l s v
                         if pl s' v ∧ ql s' then return s'
                                            else Nothing
  else if pr s v then do s' ← put r s v
                         if pr s' v ∧ qr s'
                           then maybe (return s')
                                      (const Nothing)
                                      (getBranch (pl, l, ql) s')
                           else Nothing
  else Nothing
```

Let us call *pl* and *pr* the *main conditions*, and *ql* and *qr* the *exit conditions*. The exit condition, in general, over-approximates the range of a branch. In other words, in the *get* direction, every source in the domain of a branch satisfies the

exit condition. Contrapositively, if a source does not satisfy the exit condition, then *get* for that branch will necessarily fail, and we do not need to try to execute the branch at all. This leads to the following revised definition of *getBranch*:

$$
\begin{aligned}
&\mathit{getBranch}\ (\mathit{pl}, \mathit{l}, \mathit{ql})\ \mathit{s} = \mathbf{if}\ \mathit{ql}\ \mathit{s}\ \mathbf{then\ do}\ \mathit{v} \leftarrow \mathit{get}\ \mathit{l}\ \mathit{s} \\
&\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad \mathbf{if}\ \mathit{pl}\ \mathit{s}\ \mathit{v}\ \mathbf{then}\ \mathit{return}\ \mathit{v} \\
&\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad \mathbf{else}\ \ \mathit{Nothing} \\
&\qquad\qquad\qquad\qquad\qquad\qquad \mathbf{else}\ \ \mathit{Nothing}
\end{aligned}
$$

If we do not care about efficiency, we can simply use *const True* as exit conditions, and the behavior is then exactly the same as the previous version. But if we supply disjoint exit conditions, then *get* will try at most one branch. Incidentally (but actually no less importantly), making exit conditions explicit also encourages the programmer to think about range disjointness, which is essential to totality of *Case*.

Adaptation. We have seen that, to make *Case* total, one thing we need to ensure is that the main condition of a branch should be satisfied again after the update. In practice, the main condition is usually closely related to the consistency relation, and we will only be able to deal with sources and views that are already more or less consistent; this is a rather severe restriction. As we have seen, the solution is to introduce a different kind of branches called *adaptive branches*, which can deal with sources and views that are too inconsistent by adapting the source to establish enough consistency such that a normal branch becomes applicable. Again, for simplicity, we consider only a variant of *Case* which has just one adaptive branch at the end:

$$
\begin{aligned}
&\mathbf{type}\ \mathit{CaseAdaptiveBranch}\ \mathit{s}\ \mathit{v} = (\mathit{s} \to \mathit{v} \to \mathit{Bool}, \mathit{s} \to \mathit{v} \to \mathit{s}) \\
&\mathit{Case} :: \mathit{CaseBranch}\ \mathit{s}\ \mathit{v} \to \mathit{CaseBranch}\ \mathit{s}\ \mathit{v} \to \\
&\qquad \mathit{CaseAdaptiveBranch}\ \mathit{s}\ \mathit{v} \to \mathit{BiGUL}\ \mathit{s}\ \mathit{v}
\end{aligned}
$$

The execution structure of *put* becomes slightly more complicated, as the whole thing has to be run again after adaptation; to ensure termination, we require that the second run cannot match an adaptive branch again. This is realized in BiGUL in continuation-passing style:

$$
\begin{aligned}
&\mathit{put}\ (\mathit{Case}\ \mathit{bl}\ \mathit{br}\ \mathit{ba})\ \mathit{s}\ \mathit{v} = \\
&\quad \mathit{putWithAdaptation}\ \mathit{bl}\ \mathit{br}\ \mathit{ba}\ \mathit{s}\ \mathit{v}\ (\lambda \mathit{sa} \to \\
&\qquad \mathit{putWithAdaptation}\ \mathit{bl}\ \mathit{br}\ \mathit{ba}\ \mathit{sa}\ \mathit{v}\ (\mathit{const}\ \mathit{Nothing})) \\
&\mathit{putWithAdaptation} :: \\
&\quad \mathit{CaseBranch}\ \mathit{s}\ \mathit{v} \to \mathit{CaseBranch}\ \mathit{s}\ \mathit{v} \to \mathit{CaseAdaptiveBranch}\ \mathit{s}\ \mathit{v} \to \\
&\quad \mathit{s} \to \mathit{v} \to (\mathit{s} \to \mathit{Maybe}\ \mathit{s}) \to \mathit{Maybe}\ \mathit{s} \\
&\mathit{putWithAdaptation}\ (\mathit{pl}, \mathit{l}, \mathit{ql})\ (\mathit{pr}, \mathit{r}, \mathit{qr})\ (\mathit{pa}, \mathit{f})\ \mathit{s}\ \mathit{v}\ \mathit{cont} = \\
&\quad \mathbf{if} \qquad \mathit{pl}\ \ \mathit{s}\ \mathit{v}\ \mathbf{then\ do}\ \mathit{s}' \leftarrow \mathit{put}\ \mathit{l}\ \ \mathit{s}\ \mathit{v} \\
&\qquad\qquad\qquad\qquad\qquad\qquad \mathbf{if}\ \mathit{pl}\ \mathit{s}'\ \mathit{v} \wedge \mathit{ql}\ \mathit{s}'\ \mathbf{then}\ \mathit{return}\ \mathit{s}' \\
&\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad \mathbf{else}\ \ \mathit{Nothing}
\end{aligned}
$$

```
else if pr s v then do s' ← put r s v
                       if pr s' v ∧ qr s'
                         then maybe (return s')
                                    (const Nothing)
                                    (getBranch (pl, l, ql) s')
                         else Nothing
else if pa s v then cont (f s v)
else Nothing
```

What about *get*? It turns out that *get* can simply ignore the adaptive branch! If you have doubt about this "choice", just invoke the fundamental theorem (Theorem 2): The *put* behavior is exactly what we want, and we can verify that the pair of *put* and *get* is well-behaved, so we are reassured that our "choice" is "correct", simply because there is no other choice of *get*.

To sum up, we have arrived at a simpler variant of *Case* which nevertheless has all the features of the multi-branch *Case* in BiGUL. We have inserted various dynamic checks into the *put* semantics, and the BiGUL programmer needs to be aware of these constraints to make execution of *Case* succeed: For each normal branch, (i) the main condition should be satisfied after the update, (ii) the main conditions of the branches before this one should not be satisfied after the update, and (iii) the exit condition should be satisfied by the updated source. Also the ranges of all the normal branches should be disjoint; the programmer is encouraged to write disjoint exit conditions, which imply disjointness of the ranges, and improve the efficiency of *get*. Also, for each adaptive branch, the adapted source and the view should match the main condition of a normal branch.

RearrV. Source and view rearrangements are also among the more complex constructs of BiGUL. Their complexity lies in the strongly and generically typed treatment of pattern matching, though, rather than their bidirectional behavior. The two kinds of rearrangements are fairly similar, and we will discuss view rearrangement only.

Pattern matching is inherently a bidirectional operation: In one direction, we break something into a collection of its components at the variable positions of a pattern. This collection can be considered as indexed by the variable positions, and acting like an *environment* for expression evaluation. Indeed, conversely, if we have a pattern and a corresponding environment, we can treat the pattern as an expression and evaluate it under the environment. These two directions are inverse to each other, i.e., they form an isomorphism. For the language designer, it may be slightly tedious to establish such isomorphisms, but for the programmer, pattern matching and evaluation are arguably the most natural way to decompose and rearrange things. Previous bidirectional languages usually provide theoretically simpler combinators for decomposition and rearrangement, but they are hard to use in practice. BiGUL's support of pattern matching, on the other hand, turns out to be one important contributing factor in its usability.

BiGUL's patterns are strongly typed: The programmer has to declare a target type for a pattern, and the pattern is guaranteed by typechecking to make sense for that target type. This can be achieved by defining the datatype of patterns as a generalised algebraic datatype:

$$
\begin{aligned}
&\textbf{data}\ \mathit{Pat}\ a\ \textbf{where}\\
&\quad \mathit{PVar}\ \ :: \mathit{Eq}\ a \Rightarrow \mathit{Pat}\ a\\
&\quad \mathit{PConst} :: \mathit{Eq}\ a \Rightarrow a \to \mathit{Pat}\ a\\
&\quad \mathit{PProd}\ \ :: \mathit{Pat}\ a \to \mathit{Pat}\ b \to \mathit{Pat}\ (a, b)\\
&\quad \mathit{PLeft}\ \ :: \mathit{Pat}\ a \to \mathit{Pat}\ (\mathit{Either}\ a\ b)\\
&\quad \mathit{PRight}\ :: \mathit{Pat}\ b \to \mathit{Pat}\ (\mathit{Either}\ a\ b)\\
&\quad \mathit{PIn}\ \ \ :: \mathit{InOut}\ a \Rightarrow \mathit{Pat}\ (F\ a) \to \mathit{Pat}\ a
\end{aligned}
$$

A pattern can be a (nameless) variable, a constant, a product, a *Left* or *Right* injection (for the *Either* type), or a generic constructor, and its target type is given as the index in its type. So, for example, *PProd* cannot be used to match an *Either* value. The *InOut* typeclass contains the types which can be broken into a sum-of-products representation. For example, $[a]$ is an instance of *InOut*, and $F\ [a]$, an isomorphic sum-of-products representation of $[a]$, is $\mathit{Either}\ ()\ (a, [a])$. The isomorphism is witnessed by

$$\mathit{inn} :: \mathit{InOut}\ a \Rightarrow F\ a \to a \qquad \text{and} \qquad \mathit{out} :: \mathit{InOut}\ a \Rightarrow a \to F\ a$$

These two functions will be used to define pattern matching and evaluation.

How do we define pattern matching? The result of a pattern matching, as we mentioned above, is an environment indexed by the variable positions of the pattern. Here we want a safe (but not necessarily efficient) representation of the environment type, in the sense that the indexes into the environment should be exactly the variable positions of the pattern, which is enforced statically by typechecking. In other words, this type depends on the pattern, and a way to compute this type is to encode it as a second index of the *Pat* datatype:

$$
\begin{aligned}
&\textbf{data}\ \mathit{Pat}\ a\ \mathit{env}\ \textbf{where}\\
&\quad \mathit{PVar}\ \ :: \mathit{Eq}\ a \Rightarrow \mathit{Pat}\ a\ (\mathit{Var}\ a)\\
&\quad \mathit{PConst} :: \mathit{Eq}\ a \Rightarrow a \to \mathit{Pat}\ a\ ()\\
&\quad \mathit{PProd}\ \ :: \mathit{Pat}\ a\ a' \to \mathit{Pat}\ b\ b'\ b'' \to \mathit{Pat}\ (a, b)\ (a', b')\\
&\quad \mathit{PLeft}\ \ :: \mathit{Pat}\ a\ a' \to \mathit{Pat}\ (\mathit{Either}\ a\ b)\ a'\\
&\quad \mathit{PRight}\ :: \mathit{Pat}\ b\ b' \to \mathit{Pat}\ (\mathit{Either}\ a\ b)\ b'\\
&\quad \mathit{PIn}\ \ \ :: \mathit{InOut}\ a \Rightarrow \mathit{Pat}\ (F\ a)\ b\ c \to \mathit{Pat}\ a\ b
\end{aligned}
$$

Notice that an environment type is just a product of *Var* types. We will talk about *Var* later, which is simply defined by

$$\textbf{newtype}\ \mathit{Var}\ a = \mathit{Var}\ a$$

Now we can define the pattern matching operation:

$$
\begin{aligned}
&\mathit{deconstruct} :: \mathit{Pat}\ a\ \mathit{env} \to a \to \mathit{Maybe}\ \mathit{env}\\
&\mathit{deconstruct}\ \mathit{PVar}\ \ \ x\ \ \ = \mathit{return}\ (\mathit{Var}\ x)
\end{aligned}
$$

```
deconstruct (PConst c)   x          = if c ≡ x then return () else Nothing
deconstruct (l `PProd` r) (x, y)    = liftM2 (,) (deconstruct l x)
                                                 (deconstruct r y)
deconstruct (PLeft p)    (Left x)   = deconstruct p x
deconstruct (PLeft _)    _          = Nothing
deconstruct (PRight p)   (Right x)  = deconstruct p x
deconstruct (PRight _)   _          = Nothing
deconstruct (PIn p)      x          = deconstruct p (out x)
```

and its inverse (which is total):

```
construct :: Pat a env → env → a
construct PVar          (Var x)      = x
construct (PConst c)    _            = c
construct (l `PProd` r) (envl, envr) = (construct l envl, construct r envr)
construct (PLeft p)     env          = Left  (construct p env)
construct (PRight p)    env          = Right (construct p env)
construct (PIn p)       env          = inn (construct p env)
```

Now consider view rearrangement, which evaluates a "simple" pattern-matching λ-expression on the view and continue updating with the transformed view. The body of the λ-expression refers to the variables appearing in the pattern. How do we represent such references? We have seen that an environment type is a product, i.e., a binary tree; to refer to a component in an environment, we can use a *path* that goes from the root to a sub-tree. In BiGUL, these paths are called *directions*:

```
data Direction env a where
  DVar   :: Direction (Var a) a
  DLeft  :: Direction a t → Direction (a, b) t
  DRight :: Direction b t → Direction (a, b) t
```

The type of a direction is indexed by the environment type it points into and the component type it points to. Note that the type of *DVar* is specified to work with only environment types marked with *Var*; this is for ensuring that a direction goes all the way down to an actual component at a variable position of the pattern, rather than stopping half-way, pointing to a sub-tree which include more than one component. It is easy to extract a component from an environment following a direction:

```
retrieve :: Direction env a → env → a
retrieve DVar       (Var x) = x
retrieve (DLeft  d) (x, _)  = retrieve d x
retrieve (DRight d) (_, y)  = retrieve d y
```

Now we can define *expressions*, which are similar to patterns but include directions rather than variables, to represent the body of rearranging λ-expressions:

```
data Expr env a where
  EDir   :: Direction env a → Expr env a
  EConst :: (Eq a) ⇒ a → Expr env a
  EProd  :: Expr env a → Expr env b → Expr env (a, b)
  ELeft  :: Expr env a → Expr env (Either a b)
  ERight :: Expr env b → Expr env (Either a b)
  EIn    :: (InOut a) ⇒ Expr env (F a) → Expr env a
```

Evaluating an expression under an environment is similar to inverse pattern matching:

```
eval :: Expr env a → env → a
eval (EDir d)      env = retrieve d env
eval (EConst c)    env = c
eval (l `EProd` r) env = (eval l env, eval r env)
eval (ELeft e)     env = Left  (eval e env)
eval (ERight e)    env = Right (eval e env)
eval (EIn e)       env = inn (eval e env)
```

The type of *RearrV* is then:

```
RearrV :: Pat v env → Expr env v' → BiGUL s v' → BiGUL s v
```

And its *put* behavior is simply:

```
put (RearrV p e b) s v = do env ← deconstruct p v
                            put b s (eval e env)
```

For the *get* direction, after executing the inner BiGUL program to obtain an intermediate view, we should reverse the roles of the pattern and body in the λ-expression, using the latter as a pattern to match the intermediate view. This intermediate view will be decomposed, and eventually each of its components will be paired with a direction indicating which variable position the component should go into. We can prepare a "container" shaped like the pattern, whose variable positions are initially empty. Whenever we reach a component and a direction, we try to put that component into the place in the container pointed to by the direction; if two components are put into the same position twice (indicating that the λ-expression uses a variable more than once), then they must be equal. In the end, all places in the container must be filled before it can be used as an environment to evaluate the pattern. Again, to compute the type of containers from a pattern, we add a third index to *Pat*:

```
data Pat a env con where
  PVar   :: Eq a ⇒ Pat a (Var a) (Maybe a)
  PConst :: Eq a ⇒ a → Pat a () ()
  PProd  :: Pat a a' a'' → Pat b b' b'' → Pat (a, b) (a', b') (a'', b'')
  PLeft  :: Pat a a' a'' → Pat (Either a b) a' a''
```

```
PRight :: Pat b b' b'' → Pat (Either a b) b' b''
PIn    :: InOut a ⇒ Pat (F a) b c → Pat a b c
```

A container type is just like an environment type except that the variable positions give rise to *Maybe* instead of *Var*. The first step — matching a value with an *expression* — can then be implemented as:

```
uneval :: Pat a env con → Expr env b → b → con → Maybe con
uneval p (EDir d)    x       con = unevalDir p d x con
uneval p (EConst c)  x       con = if c ≡ x then return con else Nothing
uneval p (EProd l r) (x, y)  con = uneval p l x con >>= uneval p r y
uneval p (ELeft e)   (Left x) con = uneval p e x con
uneval p (ELeft _)   x       con = Nothing
uneval p (ERight e)  (Right x) con = uneval p e x con
uneval p (ERight _)  x       con = Nothing
uneval p (EIn e)     x       con = uneval p e (out x) con

unevalDir :: Pat a env con → Direction env b → b → con → Maybe con
unevalDir PVar       DVar      x (Just y)      = if x ≡ y
                                                 then return (Just x)
                                                 else Nothing
unevalDir PVar       DVar      x Nothing       = return (Just x)
unevalDir (PConst c) _         x con           = return con
unevalDir (l `PProd` r) (DLeft  d) x (conl, conr) = liftM (, conr)
                                                    (unevalDir l d x conl)
unevalDir (l `PProd` r) (DRight d) x (conl, conr) = liftM (conl, )
                                                    (unevalDir r d x conr)
unevalDir (PLeft  p) d         x con           = unevalDir p d x con
unevalDir (PRight p) d         x con           = unevalDir p d x con
unevalDir (PIn p)    d         x con           = unevalDir p d x con
```

This function *uneval* initially takes an empty container, which is generated by:

```
emptyContainer :: Pat v env con → con
emptyContainer PVar       = Nothing
emptyContainer (PConst c) = ()
emptyContainer (l `PProd` r) = (emptyContainer l, emptyContainer r)
emptyContainer (PLeft  p) = emptyContainer p
emptyContainer (PRight p) = emptyContainer p
emptyContainer (PIn p)    = emptyContainer p
```

And then we can try to convert a container to an environment, checking whether the container is full in the process:

```
fromContainerV :: Pat v env con → con → Maybe env
fromContainerV PVar       Nothing  = Nothing
fromContainerV PVar       (Just v) = return (Var v)
fromContainerV (PConst c) con      = return ()
```

$$
\begin{array}{lll}
\mathit{fromContainerV}\ (l\ \text{`}\mathit{PProd}\text{`}\ r)\ (\mathit{conl}, \mathit{conr}) = \mathit{liftM2}\ (,) & & \\
\qquad\qquad (\mathit{fromContainerV}\ l\ \ \mathit{conl}\,) & & \\
\qquad\qquad (\mathit{fromContainerV}\ r\ \mathit{conr}) & & \\
\mathit{fromContainerV}\ (\mathit{PLeft}\ \ p) & \mathit{con} & = \mathit{fromContainerV}\ \mathit{pat}\ \mathit{con} \\
\mathit{fromContainerV}\ (\mathit{PRight}\ p) & \mathit{con} & = \mathit{fromContainerV}\ \mathit{pat}\ \mathit{con} \\
\mathit{fromContainerV}\ (\mathit{PIn}\ p) & \mathit{con} & = \mathit{fromContainerV}\ \mathit{pat}\ \mathit{con}
\end{array}
$$

If we can get hold of an environment, then we can let out a sigh of relief, since the last step — inverse pattern matching — is total. To sum up:

$$
\begin{array}{l}
\mathit{get}\ (\mathit{RearrV}\ p\ e\ b)\ s = \mathbf{do}\ v' \quad \leftarrow \mathit{get}\ b\ s \\
\qquad\qquad\qquad\qquad\qquad\quad \mathit{con} \leftarrow \mathit{uneval}\ p\ e\ v'\ (\mathit{emptyContainer}\ p) \\
\qquad\qquad\qquad\qquad\qquad\quad \mathit{env} \leftarrow \mathit{fromContainerV}\ p\ \mathit{con} \\
\qquad\qquad\qquad\qquad\qquad\quad \mathit{return}\ (\mathit{construct}\ p\ \mathit{env})
\end{array}
$$

Conceptually, this is just reversing pattern matching and expression evaluation. To actually prove the well-behavedness, though, we need to reason about stateful computation (which is what *uneval* essentially is), which involves coming up with suitable invariants and proving that they are maintained throughout the computation. It is somewhat tedious, but can be done without a problem.

It is interesting to mention that there would be a catch if we designed this combinator from the *get* direction: It is tempting to think that a rearranging λ-expression gives rise to an isomorphism, which can be lifted to a lens, and can be composed with the inner lens to give a lens semantics to RearrV. This would result in a redundant computation of an intermediate source which is immediately discarded, and now the success of the whole computation would unnecessarily depend on that of the intermediate source. To eliminate the redundant computation, we would need to use a special composition which composes a lens with an isomorphism on the right. Such a need would be hard to notice since the *get* behavior of the two compositions are the same; that is, we really have to think in terms of *put* to see that the special composition is needed.

6 Bidirectionalizing Relational Queries with BiGUL

7 Parsing and reflective printing

When we mention the *front-end* of a compiler, we usually think of a *parser* that turns concrete syntax, which is designed to be programmer-friendly and provides convenient syntactic sugar, into abstract syntax, which is concise, structured, and easily manipulable by the compiler back-end. There is another direction, though, in which a *printer* turns abstract syntax back into concrete syntax. This is useful, for example, for reporting the result of compiler optimizations done on abstract syntax to the programmer, who knows only concrete syntax. In this case, though, we would want to print the optimized program in a form that is as close to the original program as possible, so the programmer can spot what are changed — and not changed — correctly and more easily. This

is where the notion of *reflective printing* comes in: By taking both the original concrete program and the optimized abstract program as input, we can try to retain the look of the original program as much as possible. Below we will use a simplified arithmetic expression language to explain how reflective printing can be implemented in BiGUL.

7.1 Well-behavedness

It is probably obvious that the idea of reflective printing comes from *put* transformations; parsing, then, is the *get* direction. Before we proceed to implement parsing and reflective printing in BiGUL, a natural question to ask is: Is well-behavedness meaningful in the context of parsing of reflective printing? The answer is yes, especially for PutGet: An abstract syntax tree (AST) may be thought of as a concise and canonical representation of a concrete program, so it would be strange if a concrete program printed from an AST could not be parsed back to the same AST. GetPut, on the other hand, is in fact not strong enough for our purpose, as it only says that, when an AST is unmodified, printing it reflectively to the original program does not change anything, whereas we would have liked to also say that "small" changes to the AST leads to only "small" changes to the concrete program. It is at least a good start to have GetPut, though. We thus conclude that BiGUL is indeed a suitable language for implementing reflective printers and corresponding parsers.

7.2 Additive expressions

Here we use a minimal example which is simple and yet can demonstrate what reflective printing is capable of. Consider the following abstract syntax of arithmetic expressions consisting of integer constant, addition, and subtraction:

```
data Arith = Num ℤ
           | Add Arith Arith
           | Sub Arith Arith
  deriving Show
```

This is a nice representation for the compiler, but we cannot expect the programmer to write something like "*Sub* (*Num* 1) (*Add* (*Num* 2) (*Num* 3))", and should provide a concrete syntax so they can write "$1-(2+3)$". Such a concrete syntax is usually defined in terms of a BNF grammar:

```
Exp    → Exp '+' Factor
       | Exp '-' Factor
       | Factor
Factor → ℤ
       | '-' Factor
       | '(' Exp ')'
```

The two-level structure of *Exp* and *Factor* ensures that plus and minus associate to the left by default; to change association, we should use parentheses. And, to spice up the problem a little, we allow minus to be used also as a negative sign, as specified by the second production rule for *Factor*. BiGUL deals with structured data only, so we should represent a string generated using this grammar as a concrete syntax tree of the following type:

```
data Exp = Plus   Exp Factor
         | Minus Exp Factor
         | EF Factor
         | ENull
data Factor = Lit ℤ
            | Neg Factor
            | Paren Exp
            | FNull
```

Apart from the *Null* constructors, which are inserted to represent incomplete trees that can occur during reflective printing, these two datatypes are in direct correspondence with the grammar, so it is easy to recover the string from a concrete syntax tree:

```
instance Show Exp where
  show (Plus  e f) = show e ++ "+" ++ show f
  show (Minus e f) = show e ++ "-" ++ show f
  show (EF      f) = show f
  show  ENull      = "."
instance Show Factor where
  show (Lit   n) = show n
  show (Neg   f) = "-" ++ show f
  show (Paren e) = "(" ++ show e ++ ")"
  show  FNull    = "."
```

Conversely, using modern parser technologies like Haskell's `parsec` parser combinator library, we can easily implement a "concrete parser" that turns a string into a concrete syntax tree:

```
parseExp :: String → Either ParseError Exp
unsafeParseExp   :: String → Exp
unsafeParseExp s = let (Right e) = parseExp s in e
```

The rest of the job is then write a BiGUL program between *Exp* and *Arith*.

7.3 Reflective printing in BiGUL

The program is basically a case analysis: For example, when the concrete side is a plus and the abstract side is an addition, they match, and we can go into their

sub-trees recursively. For the concrete side, the right sub-tree is of type *Factor* instead of *Exp*, so in fact we will write two (mutually recursive) programs:

```
pExpArith    :: BiGUL Exp Arith
pExpArith    = Case ⊥
pFactorArith :: BiGUL Factor Arith
pFactorArith = Case ⊥
```

The branch for plus and addition can then be written as:

```
$(normalSV [p| Plus _ _ |] [p| Add _ _ |] [p| Plus _ _ |])
  ⟹ $(rearrS [| λ(Plus l r) → (l, r) |])
       $(rearrV [| λ(Add l r) → (l, r) |])
         pExpArith `Prod` pFactorArith
```

The body, which rearranges both the source and view into a product and then performs further updates on corresponding pairs, is a common programming pattern in BiGUL, so in fact there is a more compact syntax for it:

```
$(update [p| Plus l r |] [p| Add l r |]
         [d| l = pExpArith; r = pFactorArith |])
```

Following the same line of thought, we can fill in other branches to relate all abstract constructors with concrete production rules:

```
pExpArith :: BiGUL Exp Arith
pExpArith = Case
  [$(normalSV [p| Plus _ _ |] [p| Add _ _ |] [p| Plus _ _ |])
     ⟹ $(update [p| Plus l r |] [p| Add l r |]
                [d| l = pExpArith; r = pFactorArith |])
  ,$(normalSV [p| Minus _ _ |] [p| Sub _ _ |] [p| Minus _ _ |])
     ⟹ $(update [p| Minus l r |] [p| Sub l r |]
                [d| l = pExpArith; r = pFactorArith |])
  ,$(normalSV [p| EF _ |] [p| _ |] [p| EF _ |])
     ⟹ $(update [p| EF t |] [p| t |]
                [d| t = pFactorArith |])
  ]
pFactorArith :: BiGUL Factor Arith
pFactorArith = Case
  [$(normalSV [p| Lit _ |] [p| Num _ |] [p| Lit _ |])
     ⟹ $(update [p| Lit i |] [p| Num i |] [d| i = Replace |])
  ,$(normalSV [p| Neg _ |] [p| Sub (Num 0) _ |] [p| Neg _ |])
     ⟹ $(update [p| Neg t |] [p| Sub (Num 0) t |] [d| t = pFactorArith |])
  ,$(normalSV [p| Paren _ |] [p| _ |] [p| Paren _ |])
     ⟹ $(update [p| Paren t |] [p| t |] [d| t = pExpArith |])
  ]
```

This covers only "normal" cases though, namely when the source and view are "the same" except for parentheses and literals. What about the cases when the source and view have mismatched shapes? For these cases, we need adaptation. Corresponding to each branch we have already written, we add an adaptive branch which looks at the shape of the view only, throws away a mismatched source, and creates an incomplete one whose shape matches that of the view; the source will be complete created through recursive processing. For example, corresponding to the plus/addition branch, we write:

```
$(adaptiveSV [p| _ |] [p| Add _ _ |])
  ==> λ_ _ -> Plus ENull FNull
```

The final programs are:

```
pExpArith :: BiGUL Exp Arith
pExpArith = Case
  [$(normalSV [p| Plus _ _ |] [p| Add _ _ |] [p| Plus _ _ |])
     ==> $(update [p| Plus l r |] [p| Add l r |]
                  [d| l = pExpArith; r = pFactorArith |])
  ,$(normalSV [p| Minus _ _ |] [p| Sub _ _ |] [p| Minus _ _ |])
     ==> $(update [p| Minus l r |] [p| Sub l r |]
                  [d| l = pExpArith; r = pFactorArith |])
  ,$(normalSV [p| EF _ |] [p| _ |] [p| EF _ |])
     ==> $(update [p| EF t |] [p| t |]
                  [d| t = pFactorArith |])
  ,$(adaptiveSV [p| _ |] [p| Add _ _ |])
     ==> λ_ _ -> Plus ENull FNull
  ,$(adaptiveSV [p| _ |] [p| Sub _ _ |])
     ==> λ_ _ -> Minus ENull FNull
  ,$(adaptiveSV [p| _ |] [p| _ |])
     ==> λ_ _ -> EF FNull
  ]
pFactorArith :: BiGUL Factor Arith
pFactorArith = Case
  [$(normalSV [p| Lit _ |] [p| Num _ |] [p| Lit _ |])
     ==> $(update [p| Lit i |] [p| Num i |] [d| i = Replace |])
  ,$(normalSV [p| Neg _ |] [p| Sub (Num 0) _ |] [p| Neg _ |])
     ==> $(update [p| Neg t |] [p| Sub (Num 0) t |] [d| t = pFactorArith |])
  ,$(normalSV [p| Paren _ |] [p| _ |] [p| Paren _ |])
     ==> $(update [p| Paren t |] [p| t |] [d| t = pExpArith |])
  ,$(adaptiveSV [p| _ |] [p| Num _ |])
     ==> λ_ _ -> Lit 0
  ,$(adaptiveSV [p| _ |] [p| Sub (Num 0) _ |])
     ==> λ_ _ -> Neg FNull
```

```
  , $(adaptiveSV [p| _ |] [p| _ |])
      ==> \_ _ -> Paren ENull
  ]
```

7.4 Reflecting optimizations and evaluation sequences

The BiGUL programs, being bidirectional, can be executed, in the *put* direction, a reflective printer, or, in the *get* direction, as a parser. Let us look at parsing first. For example:

```
*Main> get pExpArith (unsafeParseExp "(1)+-((-1))")
Just (Add (Num 1) (Sub (Num 0) (Sub (Num 0) (Num 1))))
```

Note that negation is considered as syntactic sugar, and is desugared into a subtraction whose left operand is zero. Also note that parentheses are turned into correct structure of the abstract syntax tree, and nothing more — excessive parentheses are cleanly discarded.

For reflective printing, as we mentioned, one application is reporting what compiler optimizations do. We can replace the computation $1 - 2$ with its result -1, for example, and the reflective printer will be able to retain the excessive parentheses:

```
*Main> put pExpArith (unsafeParseExp "(1)+-((-1))")
                     (Add (Num 1) (Sub (Num 0) (Num (-1))))
Just (1)+-((-1))
```

This can also be considered as the first step of an evaluation sequence. If we keep evaluating the abstract syntax tree, the steps can all be reflected to concrete syntax:

```
*Main> put pExpArith (unsafeParseExp "(1)+-((-1))")
                     (Add (Num 1) (Num 1))
Just (1)+1
*Main> put pExpArith (unsafeParseExp "(1)+-((-1))") (Num 2)
Just 2
```

This means that if we have an evaluator on the abstract syntax, we will automatically get an evaluator on the concrete syntax!

7.5 A domain-specific language

As a final remark, the above programs may look long, but at the core of them are merely the correspondences between concrete production rules and abstract constructors. We can design a domain-specific language (DSL) that expresses such correspondences concisely, and then expand programs in this DSL into BiGUL. In fact, we have already done so, and the DSL is called *BiYacc*. For example, all the programs we have written can be generated from the following eight-line BiYacc program:

```
Arith +> Exp
Add l r +> (l +> Exp) '+' (r +> Factor);
Sub l r +> (l +> Exp) '-' (r +> Factor);
f       +> (f +> Factor);

Arith +> Factor
Num n         +> (n +> Int);
Sub (Num 0) r +> '-' (r +> Factor);
f             +> '(' (f +> Exp) ')';
```

See our draft paper for more interesting experiments about reflective printing, done on a more realistic imperative language.

References

1. Bohannon, A., Foster, J.N., Pierce, B.C., Pilkiewicz, A., Schmitt, A.: Boomerang: resourceful lenses for string data. In: POPL 2008. pp. 407–419. ACM (2008)
2. Bohannon, A., Pierce, B.C., Vaughan, J.A.: Relational lenses: a language for updatable views. In: PODS 2006. pp. 338–347. ACM (2006)
3. Fischer, S., Hu, Z., Pacheco, H.: A clear picture of lens laws - functional pearl. In: Hinze, R., Voigtl"ander, J. (eds.) Mathematics of Program Construction - 12th International Conference, MPC 2015, K"onigswinter, Germany, June 29 - July 1, 2015. Proceedings. Lecture Notes in Computer Science, vol. 9129, pp. 215–223. Springer (2015), http://dx.doi.org/10.1007/978-3-319-19797-5
4. Fischer, S., Hu, Z., Pacheco, H.: The essence of bidirectional programming. SCIENCE CHINA Information Sciences 58(5), 1–21 (2015)
5. Foster, J.: Bidirectional Programming Languages. Ph.D. thesis, University of Pennsylvania (December 2009)
6. Foster, J.N., Greenwald, M.B., Moore, J.T., Pierce, B.C., Schmitt, A.: Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem. ACM Transactions on Programming Languages and Systems 29(3), 17 (2007)
7. Hidaka, S., Hu, Z., Inaba, K., Kato, H., Matsuda, K., Nakano, K.: Bidirectionalizing graph transformations. In: ICFP 2010. pp. 205–216. ACM (2010)
8. Hofmann, M., Pierce, B.C., Wagner, D.: Symmetric lenses. In: POPL 2011. pp. 371–384. ACM (2011)
9. Ko, H., Zan, T., Hu, Z.: Bigul: a formally verified core language for putback-based bidirectional programming. In: Erwig, M., Rompf, T. (eds.) Proceedings of the 2016 ACM SIGPLAN Workshop on Partial Evaluation and Program Manipulation, PEPM 2016, St. Petersburg, FL, USA, January 20 - 22, 2016. pp. 61–72. ACM (2016), http://doi.acm.org/10.1145/2847538.2847544
10. Matsuda, K., Hu, Z., Nakano, K., Hamana, M., Takeichi, M.: Bidirectionalization transformation based on automatic derivation of view complement functions. In: ICFP 2007. pp. 47–58. ACM (2007)
11. Pacheco, H., Hu, Z., Fischer, S.: Monadic combinators for "putback" style bidirectional programming. In: PEPM '14. pp. 39–50. ACM (2014)
12. Pacheco, H., Zan, T., Hu, Z.: Biflux: A bidirectional functional update language for XML. In: Chitil, O., King, A., Danvy, O. (eds.) Proceedings of the 16th International Symposium on Principles and Practice of Declarative Programming, Kent,

Canterbury, United Kingdom, September 8-10, 2014. pp. 147–158. ACM (2014), http://doi.acm.org/10.1145/2643135.2643141
13. Voigtländer, J.: Bidirectionalization for free! (pearl). In: POPL 2009. pp. 165–176. ACM (2009)
14. Xiong, Y., Liu, D., Hu, Z., Zhao, H., Takeichi, M., Mei, H.: Towards automatic model synchronization from model transformations. In: ASE 2007. pp. 164–173. ACM (2007)